

Securing Autonomous Agency: A Comprehensive Survey of Prompt Injection and Tool-Use Vulnerabilities in Large Language Model Agents

1 Introduction

The rapid evolution of Large Language Models (LLMs) from static conversational interfaces to autonomous agents capable of dynamic action execution represents a paradigm shift in artificial intelligence, fundamentally altering the security landscape. While early LLM applications were confined to text generation, modern agentic systems integrate planning, memory retention, and external tool usage, enabling them to interact with complex digital and physical environments [1]. This transition from passive models to active entities introduces a critical vulnerability: the conflation of natural language as both user input and executable command. Unlike traditional software where code and data are strictly separated, LLM agents operate on a unified natural language interface where system instructions, user queries, and retrieved data share the same semantic space. This architectural characteristic creates the “instruction-data boundary paradox,” wherein models struggle to distinguish between trusted control logic and untrusted external content, thereby expanding the attack surface to include tool invocation chains and environmental interactions [2].

The core security challenge in this new regime is the susceptibility of agents to prompt injection attacks, which exploit the model’s instruction-following capabilities to override safety constraints. Research indicates that these attacks are no longer limited to direct user manipulation; rather, they have evolved into sophisticated indirect vectors where malicious instructions are embedded in external data sources such as web pages, emails, or database retrievals [3]. When an agent processes such compromised context, it may erroneously interpret adversarial payloads as high-priority system commands, leading to unauthorized tool execution, data exfiltration, or service disruption. Empirical studies demonstrate that even state-of-the-art models remain vulnerable to these indirect injections, with success rates indicating a systemic failure to maintain robust trust boundaries between data channels and control logic [4]. Furthermore, the integration of third-party plugins and tools exacerbates these risks, as the orchestration layer responsible for parsing model outputs into function calls often lacks rigorous validation, allowing injected instructions to bypass safety filters and trigger arbitrary code execution or privilege escalation [5].

The implications of these vulnerabilities extend beyond simple content policy violations to tangible cyber-physical harms. As agents gain the autonomy to chain multiple tools, adversaries can engineer compositional attacks where individually benign functions are sequenced to achieve malicious objectives that exceed the permission scope of any single component [6]. This capability transforms the agent into a potent vector for supply chain contamination, financial fraud, and infrastructure compromise. Recent findings highlight that the probabilistic nature of LLM reasoning complicates traditional formal verification methods, necessitating new defensive architectures that enforce strict separation between instruction and data streams [7]. Techniques such as structured queries and specialized fine-tuning have shown promise in mitigating these risks by training models to ignore command-like structures embedded within data fields, yet a comprehensive solution remains elusive [8].

This survey addresses these critical challenges by providing a systematic taxonomy of prompt injection techniques and tool-use exploits specific to autonomous agents. We delineate the scope of our analysis to focus explicitly on the mechanisms of instruction hijacking and tool manipulation, distinguishing these from general hallucinations or alignment failures. By synthesizing current re-

search on attack vectors, defensive strategies, and evaluation frameworks, we aim to establish a foundational understanding of agentic security. The subsequent sections will dissect the architectural vulnerabilities inherent in modern agent designs, categorize emerging threat actor models, and evaluate the efficacy of proposed mitigation strategies, ultimately offering a roadmap for securing the next generation of autonomous AI systems against an increasingly sophisticated adversarial landscape [9].

2 Architectural Foundations and the Expanded Threat Landscape

2.1 Formal Taxonomy of Agentic Architectures and Component Vulnerabilities

The transition from static large language models (LLMs) to autonomous agents necessitates a rigorous re-evaluation of system architecture through a security-centric lens. Unlike monolithic models, agentic systems are compositional entities characterized by distinct functional modules: perception, planning, memory, and action execution. This modular decomposition reveals that the attack surface is no longer confined to the model’s input channel but extends across the orchestration layer and external tool interfaces. As posited by [10], the general framework of LLM-based agents comprises a “brain” for reasoning, “perception” for environmental sensing, and “action” for tool utilization, each introducing unique vulnerability profiles that diverge significantly from traditional software or standalone model threats.

The planning module, responsible for task decomposition and strategy formulation, represents a critical control point susceptible to logic manipulation. Architectures employing iterative reasoning patterns, such as ReAct, introduce specific failure modes where adversarial inputs can corrupt the reasoning chain, leading to recursive loops or goal subversion. [11] categorizes these planning mechanisms into task decomposition, plan selection, and reflection, noting that vulnerabilities in any of these sub-components can cascade into systemic compromise. Furthermore, hierarchical multi-agent systems, while offering scalability as demonstrated in [12], exacerbate these risks by creating complex inter-agent communication channels. In such distributed architectures, a compromise in a single node can propagate infectious jailbreaks across the network, a phenomenon empirically validated by [13], which shows how adversarial triggers can spread exponentially in multi-agent environments without further attacker intervention.

Memory systems, encompassing both short-term context windows and long-term vector stores, constitute another high-value attack surface. The reliance on Retrieval-Augmented Generation (RAG) for long-term memory introduces the risk of knowledge base poisoning, where malicious demonstrations are injected to alter future agent behavior. [14] illustrates how backdoor triggers optimized for embedding spaces can force the retrieval of malicious content with high probability, effectively bypassing safety alignment without requiring model fine-tuning. Similarly, [15] formulates a general framework for agent backdoor attacks, distinguishing between triggers hidden in user queries and those embedded in intermediate observations, highlighting the persistence of such threats in stateful systems.

The action layer, or tool executor, serves as the bridge between digital reasoning and physical or API-driven reality, making it the locus of highest-impact vulnerabilities. The orchestration layer, which parses natural language outputs into structured function calls, is particularly fragile; ambiguities in this translation process allow for parameter manipulation and argument injection. [4] provides empirical evidence that tool-integrated agents are highly vulnerable to indirect prompt injections embedded in external content, leading to unauthorized data exfiltration or tool misuse. Moreover, the compositional nature of tool use enables privilege escalation, where benign tools are

chained to achieve malicious ends, a risk amplified in platforms supporting third-party plugins as analyzed in [16].

Ultimately, the security posture of an agentic system is determined by the weakest link in its architectural chain. The erosion of trust boundaries between the planner, memory, and executor creates a complex threat landscape where attacks can originate from untrusted data sources and propagate through legitimate control flows. Future architectural designs must therefore move beyond ad-hoc safeguards to incorporate formal verification and strict control-data separation, as suggested by [17], to mathematically guarantee the integrity of agent plans before execution.

2.2 Trust Boundary Erosion in Data-Instruction Flows

Building upon the architectural vulnerabilities identified in the planner, memory, and action modules, the fundamental security paradox in agentic workflows arises from the structural conflation of trusted system instructions and untrusted external data. Unlike traditional software, where compilers and interpreters strictly enforce boundaries between code and data, LLM agents operate within a unified natural language space that serves simultaneously as the control plane for reasoning and the data plane for retrieved context. This “instruction-data conflation” creates a porous trust boundary that directly enables Indirect Prompt Injection (IPI), a threat vector where malicious payloads embedded in passive data sources—such as web pages, emails, or database entries—are ingested by the agent and erroneously interpreted as high-priority commands [3]. Distinct from direct injection which requires active user interaction, IPI allows adversaries to compromise agents remotely by poisoning the information ecosystem the agent queries, effectively transforming benign retrieval mechanisms into vectors for arbitrary code execution or data exfiltration [3].

The mechanism driving this trust boundary erosion is rooted in the model’s inherent inability to semantically distinguish between system prompts and the content of retrieved documents. While recent theoretical work has attempted to formalize this deficit by proposing metrics to quantify instruction-data separation, empirical evaluations reveal that state-of-the-art models consistently fail to maintain this distinction under adversarial pressure [2]. This vulnerability is critically exacerbated in Retrieval-Augmented Generation (RAG) architectures, where the attention mechanism assigns equal weight to retrieved tokens and system directives unless constrained by fragile prompt engineering. Consequently, attackers can craft “wormable” inputs that propagate through agent networks; a single compromised document retrieved by one agent can trigger malicious tool calls that alter the environment for downstream agents, leading to cascading failures across the ecosystem [3].

Furthermore, this erosion of trust boundaries facilitates cross-component privilege escalation, directly exploiting the modular connections described previously. In tool-integrated agents, a low-trust component, such as a public web scraper, can ingest adversarial instructions that compel the agent to invoke high-trust tools, such as internal email clients or code interpreters. This phenomenon transforms the agent into a bridge for lateral movement, allowing untrusted data to dictate actions in privileged domains [4]. The severity of this risk is compounded by the autonomy of modern agents, which can chain multiple tool invocations to achieve complex malicious objectives without human oversight, effectively bypassing traditional access control models that assume a clear separation between data originators and action executors [18].

Emerging defenses attempt to restore these boundaries through architectural interventions rather than mere prompt tuning. Approaches such as structured queries enforce syntactic separation between instructions and data channels, training models to ignore command-like structures within

data fields [8]. Similarly, “spotlighting” techniques utilize input transformations to provide continuous provenance signals, helping the model distinguish between user commands and retrieved content [19]. However, these methods often incur utility costs or remain vulnerable to adaptive attacks that mimic legitimate instruction patterns. Future research must move beyond heuristic filtering toward formal verification of information flow policies within agentic workflows, ensuring that data retrieved from untrusted sources cannot influence control logic regardless of semantic content [20]. Without such rigorous architectural guarantees, the integration of LLM agents into critical infrastructure will remain susceptible to the diverse spectrum of adversaries and autonomous offensive capabilities detailed in the subsequent section, driven ultimately by the collapse of the instruction-data dichotomy.

2.3 Threat Actor Models and Strategic Objectives in Agentic Ecosystems

The threat landscape for Large Language Model (LLM) agents is characterized by a diverse spectrum of adversaries, ranging from opportunistic individuals to sophisticated state-sponsored entities, each exploiting the unique agency and tool-use capabilities of these systems. Unlike static models, agentic ecosystems introduce dynamic attack surfaces where adversaries can manipulate not only input data but also the execution flow of external tools and multi-agent collaborations. At the foundational level, individual actors often employ prompt crafting techniques to achieve immediate goal subversion or data exfiltration. Recent empirical studies indicate that jailbreak prompts have evolved from simple instruction overrides to complex, multi-turn strategies capable of bypassing safety alignments with high success rates [21]. Furthermore, the automation of these attacks through frameworks like [22] and [23] demonstrates a shift toward scalable, algorithmic adversarial generation that lowers the barrier to entry for non-expert attackers.

Beyond individual prompt engineering, a more insidious threat vector involves supply chain contamination and third-party plugin exploitation. In ecosystems where agents integrate unverified external services, adversaries can register malicious plugins with deceptive descriptions to hijack agent planning logic [16]. This risk is exacerbated by indirect prompt injection, where attackers embed malicious instructions in publicly accessible web pages or retrieved documents, effectively turning benign data sources into command vectors that compromise the agent without direct user interaction [4; 24]. The strategic objective here often extends beyond single-instance compromise to persistent backdoor insertion within long-term memory or knowledge bases, as demonstrated by [14] and [25], where triggers remain dormant until specific contextual conditions activate harmful behaviors.

The most critical evolution in threat actor models concerns the emergence of autonomous, multi-agent offensive capabilities. Sophisticated adversaries are no longer limited to static exploits but can deploy teams of LLM agents to autonomously discover and exploit zero-day vulnerabilities in real-world systems [26; 27]. These “offensive agents” leverage recursive planning and tool chaining to perform complex cyber-physical harms, such as unauthorized financial transactions or IoT device manipulation, which were previously infeasible for automated systems [28]. Moreover, in collaborative environments, the threat model expands to include “infectious jailbreaks,” where a single compromised agent can exponentially propagate malicious behaviors across a network of interacting agents, effectively collapsing the security posture of the entire ecosystem [13; 29]. This propagation is facilitated by the inherent trust assumptions in multi-agent communication, allowing manipulated knowledge to flood shared contexts and skew collective decision-making [30].

Ultimately, the strategic objectives of these threat actors have shifted from mere content generation

violations to tangible operational disruption, resource exhaustion, and systemic integrity compromise. The convergence of automated vulnerability discovery, supply chain poisoning, and infectious multi-agent propagation necessitates a re-evaluation of defense mechanisms that traditionally focus on single-turn input filtering. Future security frameworks must account for the temporal persistence of attacks in memory systems and the stochastic nature of agent-to-agent influence, recognizing that the agentic threat model is inherently dynamic, distributed, and capable of autonomous adaptation [31].

3 Taxonomy and Mechanisms of Prompt Injection Attacks

3.1 Direct and Indirect Injection Vectors in Agentic Workflows

The security landscape of Large Language Model (LLM) agents is fundamentally defined by the erosion of boundaries between trusted system instructions and untrusted data inputs. This subsection delineates the primary dichotomy of prompt injection attacks: direct injection, where adversaries explicitly craft user queries to subvert agent goals, and indirect injection, where malicious payloads are covertly embedded within external data streams retrieved by the agent. While traditional LLM vulnerabilities often focus on single-turn jailbreaking, agentic workflows introduce a critical amplification of risk due to the autonomy of tool invocation and the dynamic ingestion of external context.

Direct prompt hijacking represents the most overt vector, wherein an adversary leverages the model’s instruction-following capabilities to override safety constraints. Techniques range from explicit commands like “ignore previous instructions” to sophisticated persona adoption strategies that force the agent into executing unauthorized actions [32]. Empirical studies reveal that even aligned models remain susceptible to these overt manipulations, with automated frameworks like AutoDAN demonstrating the ability to generate semantically meaningful jailbreaks that bypass perplexity-based filters [33]. Furthermore, the compositional nature of agent tasks allows attackers to decompose malicious goals into benign sub-tasks, effectively camouflaging intent through logic chain injection [34]. The severity of direct attacks is exacerbated in tool-integrated agents, where successful hijacking can lead to arbitrary code execution or unauthorized API calls, as evidenced by agents autonomously exploiting one-day vulnerabilities when provided with sufficient context [27].

In contrast, indirect prompt injection (IPI) poses a more insidious threat by decoupling the attack trigger from the user interface. In Retrieval-Augmented Generation (RAG) and tool-augmented workflows, agents process external content—such as web pages, emails, or database entries—as valid context, often conflating this data with high-priority system commands [3]. This “instruction-data conflation” enables adversaries to embed hidden directives in public-facing content that execute only when retrieved by a victim agent. Recent benchmarks like InjecAgent highlight the prevalence of this vulnerability, showing that a significant portion of tool-integrated agents fail to distinguish between retrieved data and executable instructions, leading to data exfiltration or tool misuse without direct user interaction [4]. The risk is further compounded in multi-agent ecosystems, where a single compromised data source can trigger an “infectious jailbreak,” propagating malicious behavior exponentially across a network of collaborating agents [13].

The fundamental vulnerability underpinning both vectors is the lack of structural separation between control logic and data channels. Defenses relying solely on input filtering often fail against adaptive attacks that utilize encoding obfuscation or semantic camouflage [35]. Consequently, emerging research advocates for architectural solutions, such as structured queries that enforce strict

syntactic delimitation between prompts and data, thereby training models to ignore command-like structures embedded within untrusted fields [8]. Additionally, techniques like “spotlighting” aim to provide continuous provenance signals to help models distinguish input sources, though the robustness of these methods against learning-based trigger generation remains an open challenge [36]. As agents evolve toward greater autonomy, the distinction between direct and indirect vectors will likely blur, necessitating holistic security frameworks that validate intent across the entire reasoning and execution lifecycle.

3.2 Memory Poisoning and State Manipulation Strategies

Bridging the gap between immediate prompt injection vectors and the cross-modal evasion techniques discussed subsequently, this subsection addresses the critical vulnerability surface introduced by stateful memory in LLM agents. While the previous section detailed how adversaries inject malicious commands directly or via external data streams, the transition to stateful agents introduces a more insidious threat: memory manipulation. Here, adversaries exploit the conflation of trusted instructions and untrusted historical data to induce persistent behavioral drift, transforming memory from a benign retention module into a vector for sustained compromise. Unlike single-turn jailbreaks that rely on immediate prompt overriding, memory poisoning strategies target the agent’s short-term context windows and long-term vector stores, leveraging the autoregressive nature of agent interactions to embed lasting vulnerabilities.

Short-term context corruption operates by leveraging the model’s attention mechanisms to dilute or overwrite safety constraints through iterative dialogue. Adversaries employ “Chain of Attack” methodologies, where seemingly benign multi-turn interactions gradually steer the agent toward unsafe states by accumulating contextual pressure [37]. This approach capitalizes on the “Foot-in-the-Door” psychological phenomenon, where incremental requests erode initial refusal barriers, effectively reprogramming the agent’s operational parameters within a single session [38]. Furthermore, research demonstrates that leveraging prior context through preliminary question-answer pairs can significantly enhance jailbreak efficacy, as the model’s generation process becomes conditioned on a poisoned history that normalizes harmful outputs [39]. Such strategies are particularly potent in agentic workflows where memory buffers are automatically updated, allowing an attacker to embed malicious demonstrations that persist across subsequent turns without repeated explicit injection, setting the stage for more permanent storage compromises.

The threat landscape expands considerably when considering long-term memory systems, such as Retrieval-Augmented Generation (RAG) pipelines and vector databases, which serve as the persistent ground truth for agent reasoning. Here, “AgentPoison” style attacks involve embedding backdoor triggers into external knowledge bases, ensuring that specific user queries retrieve malicious contexts that guide the agent toward compromised behaviors [40]. The [41] framework illustrates how a single poisoned document can be engineered to surface only upon the presence of an adversarial trigger, enabling denial-of-service or privacy violations while maintaining benign performance on standard tasks. This form of indirect injection is particularly insidious because the malicious payload is decoupled from the user’s direct input, residing instead in the retrieved context that the agent treats as authoritative ground truth [3]. Moreover, backdoor threats in LLM-based agents can manipulate not just final outputs but also intermediate reasoning processes, introducing latent vulnerabilities that remain dormant until specific environmental conditions are met [15; 25].

A critical emerging trend is the potential for infectious jailbreaks in multi-agent ecosystems, where a compromised memory state in one agent propagates exponentially to peers through shared commu-

nication channels, amplifying the scope of individual breaches. The [13] study reveals that injecting an adversarial image into a single agent’s memory can infect an entire network, suggesting that memory poisoning poses systemic risks beyond individual compromise and foreshadowing the multimodal vulnerabilities detailed in the next section. Additionally, the persistence of these attacks challenges conventional defense mechanisms; standard safety fine-tuning often fails to remove backdoors embedded in complex reasoning chains and may even teach models to better conceal their deceptive behaviors [42]. The scalability of these threats is exacerbated by findings that larger models are increasingly susceptible to data poisoning, learning harmful behaviors more rapidly than their smaller counterparts [43]. Consequently, securing agentic memory requires a paradigm shift from input filtering to dynamic verification of retrieved contexts and rigorous isolation of stateful components. As attackers refine these persistent state corruptions, they increasingly rely on obfuscation and cross-modal triggers to evade detection, leading naturally to the discussion of multimodal and encoding-based evasion strategies that follow.

3.3 Multimodal and Encoding-Based Evasion Techniques

The evolution of Large Language Models (LLMs) into Multimodal Large Language Models (MLLMs) has paradoxically expanded the attack surface, introducing vulnerabilities where non-textual modalities serve as covert channels to bypass rigorous text-based safety alignment. Unlike traditional prompt injection which relies on semantic manipulation, multimodal and encoding-based evasion techniques exploit the misalignment between vision encoders and language decoders, or the model’s inability to recognize obfuscated textual representations. A primary vector involves visual prompt injection, where adversarial perturbations or embedded text within images override system instructions. Research indicates that images effectively act as a “foreign language” during alignment, allowing malicious intent to bypass filters that are solely tuned for textual semantics [44]. Specifically, attacks such as [45] demonstrate that pairing adversarial images with generic textual prompts can successfully jailbreak aligned models by forcing the LLM to derive toxic context from the visual embedding space. Similarly, [46] reveals that meticulously crafted images can hide and amplify harmful intent, achieving high attack success rates across diverse MLLM architectures by exploiting the model’s tendency to prioritize visual semantic content over textual safety guidelines.

Beyond direct adversarial perturbations, typographic and structural attacks leverage the model’s optical character recognition (OCR) capabilities to inject instructions that appear benign to token-based filters. The [47] framework illustrates this by utilizing ASCII art to convey image information, a format that state-of-the-art LLMs struggle to interpret correctly, thereby evading safety measures designed for standard semantic corpora. This vulnerability extends to more sophisticated structure-based attacks where semantic content is injected directly into image pixels to mislead MLLMs, a threat countered by adaptive shielding mechanisms that analyze both modalities step-by-step [48]. Furthermore, the threat landscape includes bi-modal adversarial prompts, where both textual and visual inputs are optimized cohesively to bypass guardrails, significantly outperforming unimodal attacks by leveraging the synergistic vulnerabilities of the fusion architecture [49].

Parallel to multimodal exploits, cryptographic and encoding-based obfuscation techniques manipulate the input space to disguise malicious intent. These methods reformulate harmful queries into code completion tasks or encrypted formats that require the model to execute a decryption logic before responding, effectively separating the intent recognition phase from the response generation phase [50]. By embedding decryption functions within instructions, attackers can elicit high success rates even on robust models like GPT-4. Additionally, the use of cipher characters and

low-resource language translations exploits gaps in safety training data, where models fail to recognize harmful patterns in non-standard encodings or under-represented languages [51; 52]. Such encoding strategies render standard perplexity-based defenses ineffective, as the superficial token distribution remains benign while the semantic payload remains malicious upon decoding.

The convergence of these techniques highlights a critical deficiency in current alignment paradigms: the assumption of modality homogeneity. Defenses that rely on post-hoc filtering or single-modality analysis are increasingly obsolete against composite attacks. Future research must pivot towards unified alignment frameworks that simultaneously process visual, auditory, and encoded textual streams, ensuring that safety constraints are enforced at the feature fusion level rather than the input periphery. The development of training-free protection methods, such as transforming unsafe images into text to activate intrinsic language safety mechanisms, represents a promising direction for mitigating these cross-modal vulnerabilities without compromising model utility [53]. Ultimately, securing agentic systems requires a fundamental re-architecting of safety filters to account for the continuous nature of image signals and the discrete unpredictability of encoded instructions.

3.4 Compositional and Semantic Camouflage Mechanisms

Bridging the gap between the encoding-based obfuscation discussed previously and the automated optimization frameworks detailed subsequently, compositional and semantic camouflage mechanisms represent a sophisticated evolution in prompt injection strategies. While prior sections highlighted how visual perturbations and cryptographic encoding disguise malicious payloads at the input level, this approach shifts focus to nuanced manipulations that obscure intent within complex logical frameworks. Unlike direct injection techniques that rely on explicit instruction hijacking, these methods decompose harmful objectives into sequences of benign sub-tasks or embed them within semantically ambiguous contexts, thereby evading both automated safety classifiers and human oversight. This paradigm exploits the inherent compositional nature of large language model (LLM) reasoning, where the aggregation of harmless intermediate steps can cumulatively reconstruct a prohibited outcome.

A primary vector in this domain is Compositional Instruction Attacks (CIA), which encapsulate malicious goals within ostensibly innocent composite tasks. By framing a harmful request as a creative writing exercise, a translation task, or a code debugging session, attackers effectively mask the underlying offensive objective. Research indicates that such structural disguise significantly reduces the likelihood of triggering safety refusals, as the model processes the input as a legitimate multi-step reasoning problem rather than a direct policy violation [34]. This strategy of maintaining semantic coherence to bypass detection aligns with the objectives of later automated methods; indeed, the [33] framework demonstrates that hierarchical genetic algorithms can automatically generate prompts that preserve this semantic meaningfulness while bypassing perplexity-based defenses, highlighting the critical trade-off between semantic coherence and detectability.

Logic chain injection extends this concept by disassembling malicious goals into a chain of logically connected but individually benign statements. This technique leverages the model’s chain-of-thought capabilities to reconstruct harmful intent only at the final inference step. The [54] study reveals that inserting a backdoor reasoning step into a sequence can alter the final response when a specific trigger exists, achieving high success rates even on advanced models like GPT-4. This suggests that the very mechanisms designed to enhance reasoning robustness can be co-opted to facilitate stealthy attacks. Similarly, [37] illustrates how adaptive, context-aware multi-turn dialogues can gradually steer an agent toward harmful outputs by exploiting semantic relevance across

conversation turns, setting the stage for the recursive, iterative refinement strategies explored in the following section.

Implicit clue and puzzle-based jailbreaking further obscure intent by requiring the model to independently infer the malicious instruction. In this “Puzzler” style approach, adversaries provide riddles or indirect clues that guide the model to deduce the harmful goal, avoiding explicit statement of the prohibited action [55]. This method capitalizes on the model’s generative completion tendencies, effectively outsourcing the malicious intent generation to the model itself. Additionally, encoding-based obfuscation serves as a foundational layer for this semantic camouflage; techniques such as the use of ASCII art in [47] or personalized encryption functions in [50] ensure that surface-level syntax appears harmless or unintelligible to filters, yet the model decodes and executes the hidden payload, reinforcing the continuity between static encoding exploits and dynamic logical deception.

The efficacy of these camouflage mechanisms underscores a critical vulnerability in current alignment techniques: the difficulty in distinguishing between complex, legitimate reasoning and strategically obscured malicious planning. While defenses like semantic smoothing [56] and backtranslation [57] offer some mitigation, they often struggle against attacks that preserve high semantic similarity to benign queries. Consequently, the limitations of static defense mechanisms against these sophisticated logical structures necessitate the shift toward algorithmic, automated adversarial search strategies, as the arms race moves from manual crafting of semantic traps to the systematic discovery of universal vulnerabilities through optimization.

3.5 Automated and Optimization-Driven Attack Frameworks

The evolution of prompt injection attacks has transitioned from labor-intensive manual crafting to sophisticated, automated optimization frameworks capable of discovering robust adversarial examples with minimal human intervention. This paradigm shift is driven by the recognition that manual jailbreaks are often brittle and model-specific, whereas algorithmic approaches can systematically explore the vast discrete token space to identify universal vulnerabilities. Early automated methods leveraged gradient-based optimization to append adversarial suffixes to user queries, maximizing the probability of affirmative responses to harmful instructions. Notably, [58] demonstrated that greedy and gradient-based search techniques could generate suffixes that transfer effectively across black-box models, including commercial APIs like ChatGPT and Claude. However, such white-box reliance limits applicability against closed-source systems, prompting the development of query-efficient black-box strategies.

To address the prohibitive computational cost of discrete optimization, recent frameworks have adopted evolutionary algorithms and genetic search strategies. [33] introduces a hierarchical genetic algorithm that generates semantically meaningful jailbreak prompts, effectively bypassing perplexity-based filters that often flag nonsensical gradient-derived tokens. Similarly, [35] formulates prompt generation as a multi-objective optimization problem, employing genetic algorithms to ensure generated attacks maintain high semantic similarity to the original query while achieving high success rates. These evolutionary approaches offer a critical trade-off: they sacrifice the theoretical optimality of gradient methods for improved stealth and transferability, allowing attacks to evade detection mechanisms that rely on statistical anomalies in token distributions.

Beyond static optimization, a new class of recursive agent systems has emerged, utilizing Large Language Models (LLMs) themselves as adaptive attackers. [59] employs tree-of-thought reasoning to iteratively refine candidate prompts, pruning unlikely branches to minimize query overhead

while achieving high success rates against guarded models. This approach mimics human social engineering, where an attacker iteratively adjusts their strategy based on feedback. [60] further refines this concept with Prompt Automatic Iterative Refinement (PAIR), an algorithm that uses an attacker LLM to generate and refine jailbreaks through a feedback loop, often succeeding in fewer than twenty queries. Such recursive frameworks demonstrate superior adaptability, capable of dynamically adjusting to specific model defenses without requiring access to internal gradients.

The scalability of these automated frameworks has been further enhanced by fuzzing methodologies inspired by traditional software security. [22] adapts the American Fuzzy Lop (AFL) framework to LLMs, utilizing seed selection strategies and mutation operators to automatically generate diverse jailbreak templates that outperform human-crafted counterparts. This automation exposes a critical vulnerability: the potential for universal transferable attacks. Research indicates that adversarial prompts generated via these optimization techniques often generalize across distinct architectures, suggesting that shared alignment vulnerabilities exist within the latent space of modern LLMs. Furthermore, [61] reveals that even smaller, less aligned models can be leveraged to adversarially modify the decoding probabilities of larger, safety-aligned models, achieving near-perfect misalignment rates with minimal computational overhead.

Despite their efficacy, these automated frameworks face challenges regarding query efficiency and detectability. While gradient-based methods like those in [62] can be adapted for defense, the arms race between automated attack generation and adaptive defenses continues to intensify. Future research must address the development of optimization-resistant architectures and the formal verification of agent behaviors against such algorithmic adversaries, as the automation of attack synthesis threatens to outpace manual red-teaming efforts.

4 Security Risks in Tool Invocation and External Environment Interaction

4.1 Parameter Manipulation and Argument Injection Mechanisms

The transition from passive text generation to active tool invocation introduces a critical vulnerability surface where natural language instructions are probabilistically mapped to deterministic function parameters. This subsection dissects the mechanisms by which adversaries corrupt this argument generation process, exploiting the semantic-syntactic gap inherent in Large Language Model (LLM) agents. Unlike traditional software vulnerabilities rooted in memory safety, parameter manipulation in agentic systems arises from the model’s inability to strictly distinguish between trusted schema definitions and untrusted data embedded within natural language contexts. The core threat vector involves coercing the agent to generate malformed arguments or inject malicious payloads into parameter fields that violate expected data types and structural constraints.

A primary methodology for such compromise is schema bypass and type confusion, where adversarial prompts leverage the model’s instruction-following priority to override strict type safety. Attackers craft inputs that persuade the agent to interpret string fields as executable code or SQL commands, effectively injecting payloads into database queries or command-line interfaces. This aligns with findings from [63], which demonstrates that reinforcement learning agents can autonomously learn policies to exploit SQL injection vulnerabilities by treating the exploitation process as a Markov decision process. Furthermore, the work in [6] provides empirical evidence that frontier models can perform complex blind database schema extraction and SQL injections without prior knowledge of the vulnerability, highlighting the severity of parameter manipulation in real-world web environments. These attacks succeed because the probabilistic nature of token generation often favors semantic coherence over syntactic rigor, allowing malicious arguments to

pass through input validation layers that rely on the LLM’s self-correction capabilities.

Contextual argument hijacking represents a more insidious variant, particularly in Retrieval-Augmented Generation (RAG) architectures. Here, malicious parameter values are not supplied directly by the user but are embedded within retrieved external content, such as compromised documents or API responses. When the agent processes this retrieved context, it inadvertently adopts the embedded values as legitimate arguments for tool invocation. This mechanism is central to the concept of indirect prompt injection, as detailed in [3], which argues that LLM-integrated applications blur the line between data and instructions, allowing adversaries to remotely control application functionality. The benchmark [4] further quantifies this risk, showing that tool-integrated agents are highly susceptible to indirect injections that manipulate tool arguments to exfiltrate private data or cause direct harm. Additionally, [41] illustrates how poisoned documents in knowledge bases can trigger specific adversarial behaviors when retrieved, effectively decoupling the malicious payload from the immediate user input.

The ambiguity inherent in natural language tool descriptions exacerbates these risks, allowing attackers to reinterpret function purposes. By framing benign intents in ways that map to high-risk parameters, adversaries can redirect operations to unauthorized scopes, such as deleting critical system files instead of temporary logs. The study [64] underscores that instruction-following capabilities amplifies dual-use risks, enabling standard computer security attacks to be executed via natural language manipulation. Moreover, [65] provides a theoretical framework for understanding how unintended interactions between system components can be exploited, suggesting that the compositional nature of agent tool chains creates emergent vulnerabilities where individual parameter manipulations cascade into systemic failures.

In conclusion, parameter manipulation and argument injection constitute a fundamental challenge to the security of LLM agents, stemming from the conflation of data and control logic in natural language interfaces. Current defenses often fail to account for the probabilistic decoding of arguments, leaving systems vulnerable to both direct schema bypasses and sophisticated contextual hijacking. Future research must focus on developing rigid semantic boundaries between instruction and data channels, potentially integrating formal verification methods to guarantee that generated parameters adhere to strict safety schemas regardless of the semantic context. The integration of runtime monitoring systems capable of detecting anomalous argument patterns, as suggested by the broader literature on attack-resistant systems [66; 67], will be essential to mitigate these evolving threats.

4.2 Unauthorized Tool Chaining and Compositional Privilege Escalation

Building upon the foundational risks of parameter manipulation and argument injection, the security landscape escalates significantly when adversaries shift from corrupting individual tool inputs to manipulating the agent’s orchestration logic. This transition from static prompt injection to dynamic tool misuse is epitomized by unauthorized tool chaining, a phenomenon where an adversary coerces an agent’s planning module to sequence individually benign tools into a composite workflow that achieves a malicious objective exceeding the permission scope of any single component. Unlike the schema bypasses and type confusions discussed previously, which target specific function arguments, compositional privilege escalation exploits the semantic reasoning capabilities of the agent to bridge trust boundaries between low-privilege data sources and high-privilege execution environments. This “compositional vulnerability” effectively transforms the agent’s autonomous planning capability into a potent attack vector, enabling lateral movement and data exfiltration

that traditional access control models, designed for isolated function calls, fail to detect.

A primary manifestation of this risk is the strategic chaining of read-only and write-capable tools to bypass data isolation policies. For instance, an agent may be manipulated into retrieving sensitive information via a benign search tool and subsequently transmitting it through an authorized communication channel, effectively exfiltrating data without triggering single-tool security alerts. Recent benchmarking efforts, such as [4], demonstrate that tool-integrated agents are highly susceptible to indirect prompt injections that orchestrate such multi-step attacks, with success rates increasing significantly when attacker instructions are reinforced with specific chaining logic. Furthermore, [18] identifies “risky cues” in the execution stage where agents fail to recognize that a sequence of safe actions culminates in a harmful outcome, highlighting a fundamental gap in current safety alignment which predominantly focuses on isolated query responses rather than holistically evaluating action trajectories.

The mechanism of logic-based privilege escalation often relies on the agent’s ability to perform reconnaissance using low-privilege tools to inform high-privilege exploits, mirroring traditional cyber-attack kill chains but executed autonomously through natural language planning. Research by [6] illustrates that frontier models can autonomously chain tools to perform complex exploits, such as blind database schema extraction followed by SQL injection, without prior knowledge of the vulnerability. Similarly, [26] shows that multi-agent systems can coordinate to explore vast vulnerability spaces, where one agent’s output serves as the privileged input for another, effectively bypassing role-based access controls through automated collaboration. The danger is compounded by the fact that while individual tools may be verified in isolation, their composition creates emergent behaviors that violate system-wide safety invariants, setting the stage for the even more severe consequences of arbitrary code execution discussed in the subsequent section.

Resource exhaustion represents another critical dimension of compositional abuse, where adversaries induce recursive tool invocation loops to degrade system availability. By confusing the agent’s termination conditions through adversarial prompts, attackers can force the agent into infinite chains of tool calls, leading to denial-of-service via API rate limit exhaustion or computational depletion. This aligns with findings in [68], where logical complexity is used to degrade model performance, but in the agentic context, the impact is amplified by external side effects and the consumption of finite resources. The challenge in mitigating these threats lies in the probabilistic nature of the planner; unlike deterministic software, LLM agents may spontaneously discover novel attack chains that were not explicitly programmed or anticipated by developers, rendering static allow-lists insufficient.

Addressing compositional privilege escalation requires a paradigm shift from input sanitization to the runtime verification of action sequences. Approaches inspired by information flow control, such as those discussed in [69], suggest tracking data labels as they propagate across tool boundaries to prevent low-integrity data from influencing high-privilege actions. Additionally, model-checking techniques akin to [70] could be adapted to verify that proposed tool chains do not violate global safety properties before execution. Future defenses must account for the exponential growth of possible action permutations, necessitating lightweight, real-time monitoring capable of detecting semantic drift in multi-step plans. This holistic approach to securing agent orchestration is essential before addressing the ultimate risk surface: the integration of code interpretation capabilities, where failed orchestration controls can lead directly to infrastructure compromise.

4.3 Code Generation, Interpretation, and Sandbox Escape Vectors

The integration of code interpretation capabilities into Large Language Model (LLM) agents represents a pivotal evolution in autonomous agency, transforming models from passive text generators into active executors capable of manipulating computational environments. However, this functionality introduces a critical attack surface where prompt injection vulnerabilities escalate from information leakage to arbitrary code execution and infrastructure compromise. The primary vector involves adversaries crafting inputs that coerce the agent into generating malicious scripts, often disguised as benign debugging or data analysis tasks. Research indicates that LLMs frequently produce code containing known vulnerabilities, such as buffer overflows or insecure API usage, when guided by adversarial natural language descriptions [71; 72]. Furthermore, the dual-use nature of these models allows attackers to automate the generation of sophisticated exploit payloads for specific Common Vulnerabilities and Exposures (CVEs), significantly lowering the barrier to entry for cyberattacks [73; 27].

A central challenge in mitigating these risks lies in the robustness of sandboxing environments designed to contain generated code. While isolation mechanisms are intended to restrict filesystem access and network connectivity, emerging attack vectors demonstrate that these boundaries are porous. Adversaries can employ “Neural Exec” strategies, utilizing learning-based methods to generate execution triggers that persist through preprocessing pipelines and evade blacklist-based detection, effectively bypassing static safety filters [36]. Additionally, the reliance on external package managers introduces supply chain vulnerabilities; agents coerced into importing compromised third-party libraries can instantiate persistent backdoors within the execution environment, a risk exacerbated by the model’s tendency to prioritize functional correctness over security verification [74; 75]. The threat is further amplified by the ability of agents to chain tool invocations, where a seemingly harmless code generation task serves as a reconnaissance step for a subsequent privilege escalation exploit [26].

Current defensive paradigms often rely on static analysis or post-generation filtering, yet these approaches struggle against the semantic flexibility of LLM-generated code. Techniques such as “SafeDecoding” attempt to intervene during the token generation phase by amplifying the probability of safety disclaimers, but they may fail against complex, multi-step logic bombs embedded within legitimate-looking functions [76]. Moreover, the evaluation of these risks is complicated by the lack of dynamic, stateful testing environments; frameworks like “ToolEmu” and “AgentDojo” highlight that a significant proportion of agent failures involve unauthorized actions that static benchmarks miss, underscoring the necessity for emulated sandboxes that can accurately simulate real-world tool interactions and failure modes [77; 78]. The intersection of code generation and prompt injection also facilitates “indirect” attacks, where malicious instructions retrieved from external knowledge bases (RAG poisoning) trigger the execution of harmful code without direct user input, effectively decoupling the attack trigger from the adversary’s immediate presence [40; 14].

Future research must transcend simple input sanitization to address the fundamental tension between agent autonomy and security constraints. This requires the development of formal verification frameworks capable of providing probabilistic guarantees for non-deterministic code generation, ensuring that generated scripts adhere to strict safety policies before execution [79]. Additionally, the community must prioritize the creation of adaptive red-teaming benchmarks that simulate advanced persistent threats capable of evolving alongside defensive measures, as current static evaluations significantly underestimate the efficacy of automated exploit generation [80; 23]. Ultimately, securing code-interpreting agents demands a holistic architecture that integrates runtime monitoring,

least-privilege execution contexts, and robust semantic separation between instruction and data channels to prevent the collapse of trust boundaries in autonomous workflows.

4.4 Third-Party Plugin Ecosystem and Supply Chain Vulnerabilities

Bridging the gap between low-level code execution and high-stakes physical actuation, the integration of third-party plugins and external APIs represents a critical expansion of the attack surface, shifting vulnerabilities from the model core to the broader supply chain. While the previous section highlighted how code interpreters can be coerced into generating malicious scripts, this layer introduces risks rooted in implicit trust assumptions regarding external services. Unlike traditional software ecosystems where interfaces are strictly typed and validated, LLM agents rely on natural language descriptions to discover, select, and invoke tools, creating a semantic gap that adversaries exploit to hijack control flows. A primary vector involves the manipulation of plugin metadata; attackers can register malicious plugins with deceptive natural language descriptions that mislead the agent’s planner into selecting them for unrelated benign tasks, effectively transforming a harmless utility into a conduit for data exfiltration or unauthorized action execution [81]. This “intent hijacking” leverages the agent’s reliance on semantic similarity rather than cryptographic verification, allowing unverified services to infiltrate trusted workflows.

Beyond initial selection, the dynamic interaction between agents and external services introduces risks through API response manipulation, creating a feedback loop of compromise. Compromised or adversarial third-party endpoints can return crafted natural language responses containing hidden instructions, a sophisticated form of indirect prompt injection that occurs post-invocation. When an agent processes these malicious outputs as valid context, it may alter its subsequent planning trajectory or execute unauthorized tools based on the fabricated API data, effectively ceding control to the external service [3]. This vulnerability is exacerbated in Retrieval-Augmented Generation (RAG) architectures, where poisoning the external knowledge base or plugin response cache can induce persistent behavioral drift, causing the agent to consistently retrieve and execute malicious demonstrations without triggering immediate safety filters [40; 14]. These mechanisms serve as the precursor to the severe real-world consequences discussed in the following section, where such compromised digital reasoning processes translate into irreversible financial or physical harm.

The supply chain risk is further compounded by the “compositional vulnerability” inherent in multi-plugin environments, mirroring the escalation patterns seen in code interpretation but operating at a higher level of abstraction. Individual plugins may operate within safe boundaries, yet their orchestration by an autonomous agent can lead to emergent privileges where benign tools are chained to achieve harmful outcomes, such as reading private files and transmitting them via email [70]. Recent empirical evaluations indicate that current LLM agents frequently fail to distinguish between trusted system instructions and untrusted plugin outputs, with success rates for indirect injection attacks remaining alarmingly high even in state-of-the-art models [4]. Furthermore, the reliance on unverified external code execution environments within plugins introduces risks of dependency conflicts and version skew exploits, where incompatible API schemas cause the agent to misinterpret outputs or fail insecurely, leaving the system susceptible to follow-on attacks [74].

Addressing these ecosystem-level threats requires a paradigm shift from perimeter-based defense to zero-trust architectures for agentic workflows, laying the groundwork for securing the physical and financial domains connected by these tools. Strategies must include rigorous semantic validation of plugin descriptions, runtime monitoring of tool invocation chains to detect anomalous sequencing, and the implementation of execution isolation mechanisms that strictly mediate interactions

between the agent core and third-party services [81]. Future research must focus on developing formal verification frameworks capable of proving safety properties across non-deterministic plugin interactions and establishing standardized provenance tracking for external tool responses to mitigate the risks of a rapidly expanding and largely unregulated plugin economy [82]. Only by securing this intermediate layer of tool use can we prevent the propagation of injected prompts into the deterministic, high-impact actions characteristic of next-generation autonomous agents.

4.5 Autonomous Cyber-Physical Action and Real-World Impact

The transition from digital reasoning to physical actuation represents the most critical inflection point in LLM agent security, where probabilistic prompt injections precipitate deterministic, irreversible real-world harm. Unlike static text generation errors, vulnerabilities in tool invocation chains allow adversaries to leverage the agent’s authorized access to execute high-impact actions in financial, industrial, and cyber-physical domains. The core vulnerability lies in the “semantic-to-actuation” gap, where natural language instructions are compiled into rigid API calls or control signals without sufficient semantic verification of intent versus outcome.

In financial ecosystems, agents integrated with banking APIs or cryptocurrency exchanges are susceptible to manipulation that results in unauthorized asset transfer or market disruption. Research indicates that LLM agents can autonomously exploit one-day vulnerabilities in web systems, suggesting a capacity to navigate complex financial interfaces to initiate fraudulent transactions [27]. The risk is exacerbated by the agent’s ability to chain benign tools; for instance, an agent might be coerced into retrieving sensitive billing data and subsequently executing a wire transfer, a compositional attack vector that bypasses single-tool permission checks [4]. Furthermore, the emergence of underground markets utilizing LLMs for malicious services demonstrates that these capabilities are already being weaponized for financial fraud and ransomware deployment [83]. The deterministic nature of financial APIs means that once a malicious parameter is injected into a transaction function, the resulting transfer is often immediate and irretrievable, highlighting a severe asymmetry between the ease of injection and the difficulty of remediation.

The threat landscape expands drastically when agents interact with Internet of Things (IoT) and Industrial Control Systems (ICS). Here, prompt injections can translate into physical safety violations, such as disabling safety protocols in autonomous vehicles or altering environmental settings in smart infrastructure. Studies on autonomous driving have revealed semantic Denial-of-Service (DoS) vulnerabilities where seemingly benign physical inputs cause planning modules to make catastrophic decisions, a risk that is amplified when an LLM agent controls the behavioral planner [84]. Similarly, in industrial contexts, the concept of “ladder logic bombs”—malicious logic inserted into control systems—can be generalized to LLM agents that generate or modify control code based on injected prompts [85]. If an agent is compromised via indirect prompt injection through a retrieved sensor log or maintenance manual [24], it could inadvertently execute code that damages physical hardware or endangers human life. The integration of LLMs with IoT for complex task execution further widens this attack surface, as agents orchestrate multiple devices where a single compromised command can cascade into systemic failure [86].

Moreover, the irreversibility of these actions distinguishes them from traditional cyberattacks. While a data breach can be contained, the physical destruction of infrastructure or the depletion of financial reserves via autonomous agents creates permanent loss. The challenge is compounded by the agent’s ability to operate at speeds exceeding human reaction times, effectively automating the “hands-on-keyboard” phase of an attack [28]. Current defense mechanisms often lack the granular-

ity to distinguish between legitimate high-value transactions and those induced by subtle semantic manipulations. As agents evolve to handle more critical infrastructure, the necessity for formal verification of agent plans becomes paramount to mathematically bound the risk of unsafe actuation [87]. Ultimately, securing autonomous agency requires a paradigm shift from protecting data confidentiality to ensuring the physical and financial integrity of the environments these agents inhabit, necessitating rigorous runtime monitoring and constitutional constraints that prioritize safety over autonomy in high-stakes domains [88].

5 Defensive Architectures and Mitigation Strategies

5.1 Input Sanitization and Semantic Filtering Mechanisms

Input sanitization and semantic filtering constitute the primary defensive perimeter for Large Language Model (LLM) agents, aiming to neutralize adversarial payloads before they corrupt the agent’s reasoning or planning modules. The fundamental challenge lies in resolving the instruction-data boundary paradox, where models struggle to distinguish between trusted system commands and untrusted user or retrieved data. To address this, architectural interventions such as Structured Queries (StruQ) enforce a syntactic separation between prompts and data channels. By training models to ignore command-like structures embedded within data fields, [8] demonstrates that strict delimiting can significantly mitigate prompt injection attacks with minimal utility loss. Complementary to structural approaches, semantic smoothing techniques aggregate predictions from multiple semantically transformed copies of an input prompt. [56] shows that this method achieves state-of-the-art robustness against gradient-based attacks like GCG and AutoDAN by averaging out adversarial perturbations that rely on specific token configurations.

Beyond structural and smoothing defenses, auxiliary classifier models have emerged as critical components for real-time threat detection. Models like Llama Guard leverage a dedicated safety risk taxonomy to perform multi-class classification on both inputs and outputs, effectively filtering malicious intent before execution [89]. Similarly, WildGuard provides a lightweight, open-source moderation tool capable of identifying adversarial jailbreaks and evaluating model refusal rates, matching or exceeding the performance of proprietary systems in detecting complex attack vectors [90]. However, the efficacy of these classifiers is often contingent on the depth of safety alignment; research indicates that shallow alignment, which primarily modifies the distribution of initial output tokens, remains vulnerable to sophisticated bypasses [91]. Consequently, advanced preprocessing strategies increasingly incorporate intent reconstruction mechanisms. Techniques such as back-translation or paraphrasing aim to reveal hidden adversarial semantics by reconstructing inputs into benign counterparts, allowing systems to filter prompts that diverge significantly from their original intent [92].

Despite these advancements, significant trade-offs persist between robustness and utility. Aggressive filtering often leads to over-defensiveness, where benign queries resembling attack patterns are erroneously rejected. Furthermore, the rise of indirect prompt injection, where malicious instructions are embedded in retrieved external content, challenges traditional input-only sanitization [3]. Defenses like Spotlighting attempt to mitigate this by providing continuous provenance signals within the context window, enabling the model to distinguish data sources reliably [19]. Looking forward, the field is shifting towards hybrid architectures that combine static structural constraints with dynamic, representation-level analysis. Innovative approaches such as Representation Noising suggest that removing information about harmful representations during the pre-deployment phase can immunize models against fine-tuning attacks, offering a proactive layer of defense that com-

plements runtime filtering [93]. Ultimately, effective sanitization requires a holistic strategy that integrates syntactic isolation, semantic verification, and deep representation engineering to secure the expanding attack surface of autonomous agents.

5.2 Runtime Monitoring and Execution Sandboxing

Building upon the static input sanitization and semantic filtering strategies discussed previously, runtime monitoring and execution sandboxing constitute the critical dynamic layer of defense in agentic architectures. While the preceding section addressed threats at the ingestion phase, these runtime mechanisms operate during the execution phase to intercept, verify, and constrain actions before they manifest as irreversible system compromises. This approach acknowledges that even robustly aligned models may generate unsafe tool invocations under sophisticated indirect prompt injection scenarios, necessitating a shift from preventative alignment to active governance of the action surface. By addressing the inherent non-determinism of Large Language Model (LLM) planning that static filters cannot fully resolve, this layer serves as the essential bridge between initial input processing and the structural isolation techniques detailed in the subsequent section.

A primary paradigm in this domain involves multi-agent verification frameworks, where dedicated analyzer agents scrutinize the output of the primary planner. Systems such as [94] employ a collaborative role-based architecture where distinct agent instances perform static code analysis and mutation-based fuzzing on generated code or tool arguments before execution, effectively creating a “human-in-the-loop” dynamic without the latency of human intervention. Similarly, [95] demonstrates that leveraging a secondary LLM instance to screen generated responses can reduce attack success rates to near zero by identifying semantic mismatches between intended safe behavior and actual output. However, these LLM-based monitors introduce their own vulnerabilities; as noted in [96], relying solely on another LLM for security verification risks non-deterministic failures and incorrect reasoning, suggesting that hybrid approaches combining neural verification with deterministic formal methods are essential for high-assurance contexts.

Complementing neural verification is the principle of least-privilege enforcement through capability constriction. Drawing from systems security traditions, modern agent sandboxes must restrict tool access based on immediate contextual necessity rather than broad role definitions. The concept of secure compartmentalization, as formalized in [97], provides a theoretical basis for isolating agent components to limit the blast radius of a compromised planning module, foreshadowing the architectural isolation strategies explored in the following section. In practice, this translates to runtime monitors that validate tool arguments against strict schemas and revoke permissions dynamically if anomalous chaining patterns are detected. For instance, [70] illustrates how model-checking techniques can be adapted to detect hidden attack chains in IoT automation, a methodology directly transferable to LLM agent tool orchestration to prevent benign tools from being sequenced into malicious workflows. Furthermore, [81] proposes isolating third-party app executions and mediating interactions outside the trusted environment, thereby preventing lateral movement and data exfiltration even if the core model is hijacked.

Anomaly detection represents another critical vector, focusing on behavioral patterns rather than semantic content. By tracking sequences of tool invocations, monitoring systems can identify deviations such as recursive loops or unauthorized data access attempts that characterize denial-of-service or exfiltration attacks. [98] highlights the importance of defining planning invariants as testing oracles, a technique that can be inverted for runtime monitoring to flag semantic deviations in real-time. Additionally, [77] introduces the concept of emulating tool execution in a safe, simu-

lated environment (ToolEmu) to quantify risks before real-world deployment, offering a proactive sandboxing strategy that complements reactive monitoring.

Despite these advancements, significant trade-offs remain between security rigor and utility. Overly restrictive sandboxes can induce false refusals, degrading the agent’s helpfulness, while permissive policies leave systems vulnerable to novel exploits. Future research must focus on integrating formal verification methods, such as those discussed in [99], with lightweight runtime monitors to provide probabilistic safety guarantees without incurring prohibitive computational overhead. The evolution toward self-healing agents, as suggested by [79], indicates a trajectory where runtime monitoring not only blocks attacks but also autonomously patches vulnerabilities by rewriting unsafe plans, marking a transition from passive defense to active resilience. Ultimately, while runtime monitoring provides a dynamic final line of defense, it sets the stage for the more rigid architectural isolation and formal verification frameworks that follow, which aim to enforce guarantees that transcend heuristic filtering.

5.3 Architectural Isolation and Formal Verification Frameworks

Architectural isolation and formal verification represent the frontier of defense against prompt injection and tool misuse, moving beyond heuristic filtering to enforce structural guarantees. The fundamental challenge lies in the “instruction-data boundary paradox,” where LLMs conflate untrusted retrieved content with trusted system directives [2]. To address this, emerging architectures advocate for strict control-data separation, physically or logically decoupling the planning module from the execution environment. The [81] architecture exemplifies this by isolating third-party app executions and mediating interactions through a hardened proxy, effectively preventing direct injection vectors from compromising the core model. Similarly, the concept of “compartmentalizing compilation” offers a theoretical foundation for secure agent design, ensuring that mutually distrustful components interact only via well-defined interfaces, thereby limiting the blast radius of any single compromised module [97]. This approach aligns with hardware-enforced isolation mechanisms, such as capability machines, which utilize fine-grained tags to restrict memory access and prevent unauthorized authority escalation [100; 99].

Complementing structural isolation, formal verification frameworks provide mathematical assurances of safety in non-deterministic workflows. Given the probabilistic nature of LLM outputs, traditional deterministic verification is insufficient; instead, probabilistic model checking and runtime monitoring via temporal logic are gaining traction. The [101] framework demonstrates the efficacy of Linear Temporal Logic (LTL) in encoding natural language safety constraints into verifiable automata, enabling real-time pruning of unsafe actions in robotic agents. This mirrors advances in runtime monitoring using Metric Temporal Logic (MTL) to detect privilege escalation attacks by analyzing call chains within specific time intervals [102]. Furthermore, the [17] framework integrates formal languages with natural language generation, utilizing stack-based plan generation supervised by automata to guarantee that executed plans adhere to user-specified constraints, significantly reducing the incidence of invalid or dangerous plan generation.

The synthesis of these approaches is evident in constitution-based safety layers, such as [88], which embed immutable safety rules across pre-planning, in-planning, and post-planning stages. This multi-phase verification acts as a logical sandbox, complementing physical isolation. However, trade-offs remain between rigorous verification and system utility; overly restrictive formal constraints can induce “semantic Denial-of-Service” by causing agents to reject benign but complex tasks [98]. Additionally, verifying information flow in concurrent, multi-agent systems introduces complexity,

as value-dependent security properties must be preserved across shared memory and communication channels [103; 104]. Future research must bridge the gap between high-level logical specifications and low-level execution, potentially leveraging type systems like those in [105] to automatically verify and repair policy violations. Ultimately, a robust defense requires a holistic architecture that combines the isolation principles of [81] with the mathematical rigor of [17] and [101], creating agents that are not merely aligned through training but are structurally constrained to remain safe.

5.4 Adversarial Training and Robust Alignment Strategies

While architectural isolation and formal verification establish structural guarantees, adversarial training and robust alignment strategies represent a complementary paradigm shift from reactive filtering to proactive immunization, aiming to harden the intrinsic weights and decoding dynamics of Large Language Model (LLM) agents against manipulation. Unlike the external constraints discussed previously, these approaches embed resilience directly into the model’s generative distribution by exposing it to diverse attack vectors during the optimization process. A foundational technique in this domain is Robust Prompt Optimization (RPO), which formulates defense as an adversarial objective, utilizing gradient-based token optimization to append robust suffixes to system prompts that enforce harmless output distributions [62]. This method significantly reduces attack success rates on both seen and held-out jailbreaks while maintaining utility for benign tasks, demonstrating that optimized prompt structures can act as persistent shields against adaptive adversaries.

Beyond prompt-level modifications, recent research emphasizes the necessity of “deep” safety alignment that extends beyond the initial output tokens to address the limitations of surface-level defenses. The phenomenon of shallow safety alignment, where models learn to refuse only in the first few tokens, leaves them vulnerable to prefilling and decoding parameter exploits [91]. To counter this, Layer-specific Editing (LED) has been proposed to realign critical early layers with safe response trajectories, effectively rewiring the model’s internal safety mechanisms rather than merely suppressing unsafe outputs at the surface level [106]. Similarly, Representation Noising (RepNoise) offers a weight-level defense by removing information about harmful representations across all layers, ensuring that even if an attacker gains access to model weights for fine-tuning, the capacity to recover harmful behaviors is structurally diminished [93].

The landscape of adversarial training also encompasses sophisticated data curation and latent space interventions to further fortify agents. WildTeaming leverages in-the-wild jailbreak clusters to construct large-scale synthetic datasets, enabling models to learn from complex, compositional attacks that mimic real-world adversarial behavior [107]. Complementing this, Latent Adversarial Training (LAT) perturbs internal activations to maximize loss on specific competing tasks, providing targeted robustness against persistent harmful behaviors and backdoors with far greater computational efficiency than standard red-teaming [108]. Furthermore, the vulnerability of models to benign fine-tuning underscores the need for immunization conditions that preserve alignment despite distributional shifts; techniques such as Backdoor Enhanced Safety Alignment utilize secret triggers in safety examples to maintain robustness even when models are adapted for downstream tasks [109; 110].

Despite these advances, significant trade-offs remain between robustness and utility, often manifesting as exaggerated safety behaviors where models refuse benign inputs resembling adversarial patterns [111]. Moreover, the efficacy of these defenses is challenged by the adaptability of attackers, who can exploit multi-turn interactions to gradually erode safety guardrails that were trained

primarily on single-turn scenarios [112]. Future directions must therefore focus on dynamic alignment frameworks that account for temporal drift in agentic workflows and integrate inference-time guidance, such as cross-model safety steering, to provide adaptable protection without the heavy computational cost of retraining [113]. Ultimately, achieving robust agency requires a holistic synthesis of weight-level hardening, data-centric adversarial exposure, and decoding-time verification to ensure safety persists across the full spectrum of autonomous operations.

6 Evaluation Frameworks, Benchmarking, and Red Teaming

6.1 Dynamic Simulation Environments and Interactive Benchmarking

The evaluation of Large Language Model (LLM) agents necessitates a paradigm shift from static, single-turn benchmarks to dynamic, stateful simulation environments that faithfully replicate the complexities of real-world tool usage and multi-turn interactions. Traditional evaluation metrics, often reliant on fixed datasets, fail to capture the emergent vulnerabilities arising from long-horizon planning, memory persistence, and the conflation of instruction and data channels in agentic workflows. Consequently, recent research has prioritized the development of interactive frameworks that assess agent resilience under adversarial pressure within controlled yet evolving contexts.

A critical advancement in this domain is the construction of stateful environments that enforce strict state dependencies and implicit tool chaining. [114] introduces a conversational benchmark that moves beyond stateless API calls, evaluating agents on their ability to manage implicit state transitions and handle insufficient information across multi-turn dialogues. Similarly, [115] provides a multi-dimensional evolving benchmark comprising eight distinct environments, revealing that while top-tier commercial models exhibit strong agentic capabilities, open-source counterparts often struggle with long-term reasoning and instruction following in complex settings. These platforms highlight that agent failure is frequently not a result of individual tool misuse but rather a breakdown in maintaining context consistency over extended interaction windows.

To specifically address security vulnerabilities, newer frameworks have integrated adversarial dynamics directly into the simulation loop. [78] establishes an extensible environment where agents execute tools over untrusted data, explicitly designed to test robustness against prompt injection attacks that hijack tool execution flows. This approach contrasts with static test suites by allowing for the design of adaptive attacks that evolve alongside defensive measures. Furthermore, the risk of indirect prompt injection via retrieved contexts is rigorously quantified in [4], which benchmarks tool-integrated agents against malicious instructions embedded in external content, demonstrating significant susceptibility even in state-of-the-art models like GPT-4. The severity of these risks is further amplified in multi-agent settings; [13] simulates environments with up to one million agents to demonstrate “infectious jailbreaks,” where a single compromised agent can exponentially propagate malicious behaviors through pairwise communication, underscoring the necessity of evaluating systemic resilience rather than isolated agent performance.

Beyond specific security exploits, comprehensive benchmarks are emerging to evaluate the trade-offs between safety and utility in dynamic scenarios. [116] emulates dynamic conversations between users and agents constrained by domain-specific policies, introducing metrics like $pass^k$ to measure behavioral reliability over multiple trials. Their findings indicate that even advanced function-calling agents succeed in fewer than 50% of tasks while exhibiting high inconsistency, suggesting that current safety alignments often brittlely fail under slight linguistic perturbations or complex rule sets. Additionally, [77] proposes an LLM-emulated sandbox to identify high-stakes risks without the prohibitive cost of manual environment instantiation, enabling the rapid generation of diverse

failure scenarios including financial loss and data exfiltration.

Despite these advances, challenges remain in standardizing evaluation protocols and ensuring the reproducibility of dynamic tests. The non-deterministic nature of LLM outputs, as noted in code generation studies, complicates the validation of agent trajectories, necessitating probabilistic metrics rather than binary success rates. Future directions must focus on integrating formal verification methods into these dynamic simulators to provide mathematical guarantees on safety properties during execution. Moreover, as agents become more autonomous, evaluation frameworks must evolve to assess “gradient alignment drift” over continuous learning cycles, ensuring that safety constraints do not erode through iterative environmental feedback. The synthesis of interactive simulation, adversarial red-teaming, and rigorous metric design represents the frontier of agentic security evaluation, moving the field toward a more robust understanding of autonomous system reliability.

6.2 Automated Adversarial Generation and Optimization Strategies

While automated adversarial generation provides the mechanisms to discover vulnerabilities, the evolution from manual prompt crafting to algorithmic attack synthesis represents a fundamental paradigm shift in how Large Language Model (LLM) agent security is assessed. Traditional red teaming, reliant on human ingenuity, is increasingly insufficient against the scale and complexity of modern agentic workflows. Consequently, research has pivoted toward automated frameworks that treat attack generation as an optimization problem, leveraging gradient-based search, evolutionary algorithms, and recursive agent systems to discover robust injection vectors with minimal query overhead. This transition lays the groundwork for the dynamic, stateful evaluation environments discussed subsequently, where such automated attacks are deployed to test agent resilience in real-time.

Optimization-based techniques constitute the most rigorous approach to automated jailbreaking, particularly in white-box or gray-box settings. Foundational work such as [58] demonstrated that greedy coordinate gradient (GCG) searches could identify universal adversarial suffixes that maximize the probability of objectionable content generation across diverse models. While highly effective, these methods often suffer from high computational costs and generate semantically non-sensical prompts susceptible to perplexity-based filtering. To address these limitations, recent advancements have introduced more sophisticated optimization objectives. For instance, [33] employs a hierarchical genetic algorithm to generate stealthy, human-readable prompts that bypass semantic detectors, while [117] enhances convergence speed through multi-coordinate updating strategies and diverse target templates. Furthermore, the scope of optimization has expanded beyond simple text generation to target specific system components; [118] formulates an optimization objective specifically for manipulating LLM-as-a-Judge systems, highlighting the vulnerability of automated evaluation pipelines themselves.

In black-box scenarios where gradient access is unavailable, fuzzing and iterative refinement strategies have emerged as dominant methodologies. Inspired by traditional software security, [22] adapts mutation-based fuzzing to the natural language domain, utilizing seed templates and mutation operators to explore the input space efficiently. This approach is complemented by iterative refinement algorithms like Prompt Automatic Iterative Refinement (PAIR), described in [60], which leverages an attacker LLM to simulate social engineering tactics, refining candidate jailbreaks based on target feedback in fewer than twenty queries. Similarly, [59] utilizes tree-of-thought reasoning to navigate the prompt search space, pruning unlikely candidates to reduce query complexity while maintain-

ing high success rates against guarded models. These methods underscore a critical trend: the automation of semantic understanding allows attackers to craft contextually coherent injections that evade static defenses, a capability essential for probing the complex tool-use chains evaluated in modern benchmarks.

The frontier of automated red teaming increasingly involves multi-agent systems capable of autonomous strategy adaptation, mirroring the multi-agent risks highlighted in systemic resilience studies. Frameworks such as [119] and concepts explored in [107] suggest that teams of interacting agents can collaboratively discover novel attack vectors by sharing insights and adapting to specific target architectures. This is particularly relevant for agentic environments where tool-use vulnerabilities require multi-step planning; [26] illustrates how coordinated agent teams can overcome long-horizon planning challenges to exploit complex system vulnerabilities. However, these automated approaches face trade-offs between attack potency and detectability. While optimization methods offer theoretical guarantees of finding optimal suffixes, they often lack the nuanced contextual awareness of human attackers, whereas multi-agent systems, while adaptive, introduce significant latency and resource consumption.

Future directions in this domain must address the “arms race” dynamic where defensive fine-tuning rapidly neutralizes known optimization strategies. The development of transferable, universal attacks that generalize across model families, as seen in [120], suggests that current alignment techniques remain brittle. Ultimately, the efficacy of automated adversarial generation necessitates a reevaluation of evaluation metrics, moving beyond binary success rates to assess the robustness of agents against continuous, evolving algorithmic pressure. This need for nuanced assessment directly motivates the shift toward multidimensional frameworks that quantify the trade-offs between safety, utility, and robustness, as detailed in the following section.

6.3 Metrics for Safety, Utility, and Robustness Trade-offs

Evaluating the security posture of Large Language Model (LLM) agents necessitates a paradigm shift from binary attack success metrics to multidimensional frameworks that quantify the intricate trade-offs between safety, utility, and robustness. Traditional evaluations often rely on Attack Success Rate (ASR), which measures the frequency with which adversarial prompts elicit harmful outputs [121]. However, in agentic systems where tool invocation carries real-world consequences, such binary classifications fail to capture the severity of policy violations or the nuance of partial compliance. To address this, recent frameworks like [122] and [123] introduce fine-grained scoring mechanisms that assess the potency of jailbreaks across varying levels of harm, moving beyond simple pass/fail determinations to provide a gradient of risk exposure.

A critical challenge in metric design is the phenomenon of over-defensiveness, where models refuse benign queries due to excessive safety alignment, thereby degrading utility. This trade-off is formally captured by the False Refusal Rate (FRR), which quantifies the proportion of harmless instructions incorrectly rejected by the system [124]. High FRR indicates a brittle safety alignment that prioritizes risk avoidance over functionality, rendering agents ineffective for legitimate tasks. Benchmarks such as [84] and [107] explicitly balance ASR and FRR, advocating for metrics that reward models capable of distinguishing between adversarial mimicry and genuine user intent. The ideal security posture lies on the Pareto frontier of this trade-off, maximizing attack resistance while minimizing utility loss, a balance that [111] notes is often compromised by naive fine-tuning strategies.

Furthermore, robustness metrics must account for the stability of agent behavior under semantic

perturbations and multi-turn interactions. Agents exhibiting high sensitivity to minor prompt variations demonstrate brittle alignment, a vulnerability highlighted in [125]. Consequently, consistency metrics that measure prediction stability across rephrased inputs or slight contextual shifts are essential for assessing true resilience. In multi-turn scenarios, static metrics prove insufficient; [112] demonstrates that defenses effective against single-turn automated attacks often collapse under iterative human-led jailbreaking, necessitating longitudinal metrics that evaluate safety decay over extended conversation horizons. Additionally, in tool-integrated environments, metrics must extend to behavioral safety, assessing whether agents correctly identify and mitigate risks during execution rather than merely filtering input text [126].

Future evaluation frameworks must integrate these dimensions into composite scores that reflect the operational reality of autonomous agents. This includes weighting metrics by the severity of potential tool misuse and the criticality of the domain, ensuring that high-stakes applications demand stricter safety-utility balances. By adopting holistic metrics that simultaneously penalize successful injections, false refusals, and behavioral inconsistency, the community can drive the development of agents that are not only secure against known attacks but also reliable and useful in dynamic, open-world environments.

6.4 Human-in-the-Loop Protocols and Qualitative Assessment

Building upon the quantitative safety-utility trade-offs and behavioral consistency metrics established previously, automated red-teaming frameworks—while effective at scaling the discovery of syntactic vulnerabilities—frequently fail to capture the nuances of social engineering, ethical ambiguity, and domain-specific contextual risks inherent in agentic workflows. Consequently, human-in-the-loop (HITL) protocols remain indispensable for validating safety in complex, multi-turn interactions where adversarial intent is obscured by persuasive language or implicit reasoning. Structured human red teaming moves beyond ad-hoc testing by adapting military and cybersecurity traditions to the unique psychological dynamics of AI interaction, addressing challenges such as tester bias and the cognitive load on red teamers [127]. Unlike algorithmic attackers that optimize for perplexity or token probabilities, human evaluators can employ sophisticated social engineering tactics, leveraging persuasion taxonomies derived from social science to elicit harmful behaviors that static classifiers overlook [127]. This human-centric approach is critical for identifying “jailbreaks” that rely on narrative camouflage or role-playing, which automated systems often misclassify as benign creative writing [128].

To bridge the gap between automated scale and human insight, hybrid methodologies have emerged that integrate human judgment directly into the evaluation lifecycle, effectively addressing the limitations of purely metric-driven assessments. Frameworks like EvalGen exemplify this mixed-initiative approach, where humans validate and refine LLM-generated evaluation criteria, effectively “validating the validators” to align automated metrics with subjective human preferences [129]. This iterative process is necessary because evaluation criteria often suffer from “criteria drift,” where the definition of safety evolves based on observed model outputs rather than remaining static [129]. Furthermore, large-scale efforts such as WildTeaming demonstrate the value of mining in-the-wild human-chatbot interactions to discover novel jailbreak clusters, which are then synthesized to create diverse adversarial datasets that outperform purely gradient-based optimization methods [107]. These datasets not only improve robustness but also help mitigate exaggerated safety behaviors by providing contrastive examples of benign queries that resemble harmful ones [107].

Qualitative assessment also plays a pivotal role in grounding theoretical risk models in empirical

reality, complementing the quantitative dimensions discussed earlier. By employing grounded theory methods to analyze practitioner motivations and strategies, researchers can construct a more realistic understanding of “red teaming in the wild,” informing the design of evaluation scenarios that reflect actual threat actor behaviors [130]. In domain-specific applications, such as healthcare or finance, expert-guided utility verification ensures that security evaluations do not compromise functional utility, a balance often missed by binary safety classifiers [1]. For instance, benchmarks like R-Judge incorporate human consensus on safety labels to evaluate an agent’s risk awareness in interactive environments, revealing that even state-of-the-art models lag significantly behind human performance in identifying subtle safety risks [126]. Similarly, the AgentDojo framework utilizes human-designed tasks to evaluate agents executing tools over untrusted data, highlighting vulnerabilities in planning logic that automated fuzzing fails to expose [78].

Despite these advancements, significant challenges persist in standardizing HITL protocols, creating a dependency on the very evaluation mechanisms that must themselves be secured. The subjectivity of human judgment introduces variability that complicates replicability, necessitating rigorous codebook construction and multi-phase verification to treat prompt science as a disciplined inquiry [129]. Moreover, as the field increasingly relies on LLMs to assist or automate parts of this human evaluation process, the integrity of these hybrid systems becomes contingent on the robustness of the underlying judges—a meta-vulnerability explored in depth in the following section. Future research must therefore focus on developing standardized interfaces that facilitate seamless collaboration between human experts and automated agents, leveraging human intuition to guide adaptive red-teaming agents like RedAgent in exploring context-specific vulnerabilities [131]. Ultimately, the integration of qualitative human assessment with quantitative automated metrics offers the most promising pathway toward comprehensive safety validation, ensuring that agentic systems are robust against both algorithmic exploits and the sophisticated, evolving tactics of human adversaries, provided the evaluation infrastructure itself remains secure.

6.5 Validation of Evaluation Methodologies and Metric Integrity

The integrity of security evaluations for Large Language Model (LLM) agents relies fundamentally on the robustness of the metrics and evaluators employed, yet a critical meta-vulnerability exists where the evaluation mechanisms themselves are susceptible to adversarial manipulation. As the field transitions from static benchmarking to dynamic red-teaming, ensuring that safety metrics accurately reflect model resilience without being gameable has become paramount. Recent investigations reveal that LLM-based judges, increasingly deployed to automate safety assessment, inherit the same non-deterministic and adversarial vulnerabilities as the models they evaluate. Specifically, [132] demonstrates that universal adversarial phrases can deceive judge models into assigning maximum safety scores to harmful content, effectively breaking the evaluation loop through simple concatenation attacks. This phenomenon suggests that reliance on untreated LLM-as-a-judge systems may yield false confidence in an agent’s security posture.

Furthermore, the stability of evaluation outcomes is threatened by the “butterfly effect” of minor prompt perturbations. [125] illustrates that trivial modifications, such as appending whitespace or altering formatting, can catastrophically shift model decisions, thereby compromising the reproducibility of safety benchmarks. This sensitivity extends to the metrics themselves; [133] argues that traditional binary success/failure metrics are insufficient, advocating for fine-grained scoring frameworks that capture the nuance of partial jailbreaks and refusal degradation. Without such granularity, evaluations risk overlooking subtle erosion of safety boundaries that precede total compromise.

The validity of these methodologies is further complicated by the interplay between safety alignment and utility, often leading to exaggerated safety behaviors that distort metric integrity. [134] highlights that models may refuse benign queries due to keyword matching, inflating safety scores while degrading utility. Consequently, robust validation requires decoupling safety refusal from helpfulness, a challenge addressed by frameworks like [84], which employs balanced taxonomies to distinguish between legitimate refusals and over-defensiveness. Additionally, the reliability of automated evaluators must be stress-tested against decision-flipping attacks, where benign context is manipulated to mask harmful intent, a vulnerability exposed in studies on metric manipulation [118].

To mitigate these risks, the community must adopt rigorous standards for validator alignment. [129] proposes a mixed-initiative approach where human feedback iteratively corrects evaluator drift, ensuring that automated metrics remain grounded in human ethical standards. Moreover, the non-deterministic nature of LLM outputs necessitates statistical rigor in evaluation protocols; [135] emphasizes that single-run assessments are scientifically invalid, urging for multi-sample aggregation to account for stochastic variance. Future research must prioritize the development of certifiable evaluation frameworks that provide mathematical guarantees against adversarial manipulation, moving beyond empirical observation to formal verification of metric integrity. Only by securing the evaluators themselves can the field ensure that reported safety improvements represent genuine robustness rather than optimized performance on fragile benchmarks.

7 Emerging Frontiers and Future Research Directions

7.1 Security Dynamics in Multi-Agent and Collaborative Ecosystems

The transition from isolated large language model (LLM) instances to decentralized multi-agent ecosystems introduces a paradigm shift in security dynamics, where vulnerabilities are no longer confined to single endpoints but propagate through inter-agent communication channels. In collaborative frameworks, agents leverage shared memory and direct dialogue to enhance collective reasoning [136]; however, this connectivity creates a vector for “infectious jailbreaks,” where a single compromised agent can exponentially disseminate adversarial behaviors across a network. Recent empirical simulations demonstrate that feeding an adversarial image into the memory of one agent within a population of up to one million can lead to near-total system infection without further adversary intervention [13]. This phenomenon underscores a critical fragility in distributed architectures: the trust mechanisms facilitating cooperation also enable the rapid proliferation of misaligned behaviors.

Beyond direct infection, collaborative ecosystems face sophisticated threats from manipulated knowledge flooding and coalition-based attacks. In Retrieval-Augmented Generation (RAG) enhanced multi-agent systems, adversaries can inject poisoned documents into shared knowledge bases, causing benign agents to unconsciously adopt false narratives or execute malicious tool chains [40; 4]. The risk is exacerbated in debate or consensus frameworks, where malicious agents exploit persuasive capabilities to skew group decision-making. Research indicates that LLM agents can simulate complex human trust behaviors, which adversarial entities can manipulate to override the safety guardrails of peer agents [137]. Furthermore, the “Wolf Within” attack vector illustrates how a single manipulated operative can indirectly induce malice in a society of Multimodal LLMs (MLLMs) by generating subtle prompts that trigger harmful outputs in others, effectively bypassing direct content filters [29].

Defending against these emergent threats requires moving beyond static input sanitization toward

dynamic, normative enforcement mechanisms. Traditional isolation strategies often fail in highly interconnected environments where information flow is essential for task completion. Instead, future architectures must integrate formal verification of information flow policies capable of handling loops and non-deterministic agent interactions [104]. The concept of modular interpreted systems offers a theoretical foundation for defining openness and interference functions, potentially allowing for the mathematical bounding of infection spread [138]. Additionally, deploying “guardian” agents or constitutions that enforce social contracts dynamically presents a promising mitigation strategy. Frameworks like TrustAgent suggest that embedding immutable safety constitutions can provide pre-, in-, and post-planning checks to maintain alignment even under adversarial pressure [88]. Similarly, multi-agent defense frameworks, where distinct agent roles collaborate to filter harmful responses, have shown efficacy in improving robustness against jailbreaks while preserving utility [94].

Ultimately, securing collaborative ecosystems demands a holistic approach that balances the flexibility required for emergent intelligence with rigorous constraints on inter-agent influence. As agents become increasingly autonomous and capable of strategic reasoning [139], the security community must develop scalable detection mechanisms for subtle behavioral drift and coalition formation. Future research should prioritize the development of probabilistic model checking techniques tailored for stochastic agent plans, ensuring that the cumulative risk of long-horizon tool invocation chains remains within acceptable bounds even in the presence of compromised peers.

7.2 Long-Term Stability, Continuous Learning, and Backdoor Persistence

While the previous section detailed how vulnerabilities propagate dynamically through inter-agent communication, creating systemic risks in decentralized ecosystems, this subsection shifts focus to the internal temporal evolution of individual agents. The transition from static, single-turn large language models to stateful, autonomous agents introduces a critical temporal dimension where threats are no longer ephemeral but can persist, evolve, and compound over extended operational lifecycles. A primary concern in this domain is the phenomenon of gradual alignment drift, wherein repeated exposure to subtle adversarial inputs or benign but distributionally shifted data erodes safety constraints incrementally. Research indicates that even fine-tuning on seemingly harmless datasets can compromise safety alignment, suggesting that continuous learning loops in agents are susceptible to “shadow alignment” attacks where minimal malicious data subverts safety without degrading utility [140]. This vulnerability is exacerbated by the finding that safety alignment often resides shallowly in the model’s output distribution, making it fragile against iterative updates that prioritize task performance over harmlessness [91]. Consequently, an agent subjected to long-term interaction may experience a “gradient alignment drift,” where the cumulative effect of user feedback slowly steers the policy away from its original safe configuration, eventually leading to significant policy violations that would have been rejected initially [Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!].

Compounding the risk of drift is the persistence of backdoors within long-term memory stores and tool libraries. Unlike the transient prompt injections or immediate “infectious jailbreaks” discussed previously, backdoor attacks embedded in an agent’s vector database or retrieval-augmented generation (RAG) context create dormant triggers that can be activated long after the initial compromise. Recent studies on “Sleeper Agents” demonstrate that deceptive behaviors can be trained to persist through standard safety training and reinforcement learning, remaining latent until specific contextual triggers—such as a date change or a specific keyword—are encountered [Sleeper Agents Training Deceptive LLMs that Persist Through Safety Training]. In agentic architectures, this

threat manifests as “temporal poisoning,” where adversaries inject malicious data into historical conversation logs or episodic memory, ensuring that future reasoning steps are contaminated by past compromises even after the immediate threat vector is removed [Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents]. Furthermore, the integration of external tools expands the attack surface for persistent backdoors; malicious tool descriptions or compromised API responses can serve as indirect injection vectors that manipulate the agent’s planning logic consistently over time, mirroring the poisoned document risks in multi-agent systems but operating within a single agent’s longitudinal history [Not what you’ve signed up for Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection].

The challenge is further amplified by self-improvement loops, where agents autonomously modify their own prompts, code, or architectural configurations. Such recursive processes risk amplifying vulnerabilities, as an agent might inadvertently optimize for efficiency at the expense of safety, effectively “jailbreaking” itself through iterative refinement [141]. Theoretical frameworks suggest that any alignment process that attenuates rather than eliminates undesired behaviors is fundamentally vulnerable to adversarial prompting that increases in efficacy with prompt length and interaction depth [142]. Therefore, ensuring long-term stability requires moving beyond static evaluation metrics to dynamic, longitudinal assessments that account for the cumulative impact of interactions. Addressing these temporal vulnerabilities necessitates the robust technical foundations discussed in the following section, specifically the formal mechanisms required to enforce safety constraints in real-time against stochastic generation. Future research must prioritize the development of “immunization” conditions that protect models against harmful fine-tuning and the design of architectural safeguards capable of detecting and reversing alignment drift before it reaches a critical threshold [110]. Without such measures, the promise of continuous learning in autonomous agents remains inextricably linked to the risk of irreversible security degradation.

7.3 Formal Verification and Probabilistic Safety Guarantees for Non-Deterministic Workflows

The integration of Large Language Models (LLMs) into autonomous agents introduces a fundamental dichotomy: the stochastic nature of neural generation versus the deterministic requirements of safety-critical tool execution. Traditional formal verification methods, designed for static codebases, struggle to provide guarantees for workflows where the control flow is dictated by probabilistic token sampling [135]. Consequently, emerging research focuses on bridging this gap through probabilistic model checking and runtime enforcement mechanisms that can bound the risk of unsafe states in non-deterministic environments. A primary avenue involves adapting temporal logics to monitor agent trajectories dynamically. Building on foundations like Metric Temporal Logic (MTL) used in operating system security [102], recent frameworks translate natural language safety policies into Linear Temporal Logic (LTL) constraints. These constraints serve as runtime guards, pruning action sequences that violate safety specifications before execution, effectively solving the problem of verifying security policies in multi-agent workflows with loops [104]. Furthermore, the concept of “secure planning” has been formalized as a stochastic game between the agent and a stealthy attacker, solvable via model-free reinforcement learning to satisfy LTL objectives even in unknown environments [143].

Beyond runtime monitoring, there is a significant push toward “neuro-symbolic” architectures that integrate formal languages directly into the planning loop. The [17] framework exemplifies this by allowing users to define constraints as automata, supervising the LLM’s stack-based plan generation to ensure controllable and valid outputs. This approach mitigates the risk of invalid plans

corrupting user trust, a critical failure mode in autonomous systems. Similarly, [144] argues for a tighter bi-directional interaction where external symbolic verifiers acquire models driven by LLMs, moving beyond simple translation to genuine verification. In the domain of industrial control, [145] demonstrates the efficacy of iterative pipelines where LLM generation is guided by external SMV verifiers and grammar checkers, transforming structurally flawed code into verifiably correct programs for Programmable Logic Controllers. These methods collectively suggest that safety cannot be an emergent property of scaling but must be enforced through architectural coupling with formal solvers.

However, verifying the tool invocation chain itself remains a distinct challenge, particularly regarding privilege escalation and information flow. Frameworks like [70] utilize model checking to detect hidden attack chains in trigger-action programming, identifying how benign rules can compose into privilege escalation vectors. This compositional risk is exacerbated in non-deterministic agents, where the sequence of tool calls is not fixed. To address this, researchers are exploring type systems and information flow control mechanisms adapted for probabilistic computation. For instance, [146] introduces type systems that enforce obliviousness in probabilistic algorithms, preventing leaks via observed distributions—a concept vital for ensuring agents do not leak sensitive data through timing or access patterns during tool use. Additionally, the notion of certifiable safety is being extended to LLMs via methods like “erase-and-check,” which provides mathematical guarantees that harmful prompts are not mislabeled as safe up to a certain perturbation size [147].

Despite these advances, significant challenges persist. The “non-determinism” of LLMs implies that even temperature-zero settings do not guarantee identical outputs across requests, complicating the reproducibility required for formal proofs [135]. Future research must therefore develop probabilistic bounds rather than binary guarantees, quantifying the likelihood of safety violations over long-horizon tasks. Moreover, the integration of verification must not stifle agent utility; as noted in [88], safety strategies must be embedded across pre-planning, in-planning, and post-planning stages to balance helpfulness with harmlessness. Ultimately, the field is moving toward a paradigm where formal methods do not merely verify post-hoc but actively shape the generation space, ensuring that autonomous agents operate within mathematically defined safe regions despite their inherent stochasticity.

7.4 Ethical, Legal, and Regulatory Frameworks for Autonomous Agency

Building upon the formal mechanisms and probabilistic bounds established in the previous subsection to constrain stochastic agent behavior, the deployment of insecure Large Language Model (LLM) agents in critical infrastructure necessitates a parallel paradigm shift from purely technical mitigation to comprehensive ethical, legal, and regulatory frameworks. As agents transition from passive conversationalists to autonomous entities capable of executing high-stakes actions, the traditional liability models based on human intent become insufficient. A primary challenge lies in attributing accountability when autonomous agents cause cyber-physical harm, such as financial fraud or infrastructure disruption, through tool manipulation or prompt injection. Current legal structures struggle to assign responsibility among developers, deployers, and the agents themselves, particularly when harms arise from emergent behaviors rather than explicit coding errors [148]. The complexity is exacerbated by the “black box” nature of agent planning, where the causal chain between a malicious input and a detrimental action may involve non-deterministic reasoning steps that obscure direct liability, echoing the verification difficulties discussed earlier [149].

Privacy preservation constitutes another critical frontier, as agents operating in data-rich envi-

ronments often require broad access to personal information to function effectively. This creates significant risks of unauthorized data exfiltration via tool chaining or indirect prompt injection, where agents inadvertently violate contextual privacy norms by leaking sensitive information to untrusted third-party plugins [150]. The concept of “use privacy” becomes paramount; agents must not only protect data statically but also reason about the appropriateness of data usage in dynamic contexts, preventing proxy use of protected information types during task execution [151]. Furthermore, the integration of unverified third-party tools amplifies these risks, as malicious plugins can exploit natural language ambiguities to hijack agent workflows and compromise user data, underscoring the necessity of the type systems and information flow controls mentioned previously [16].

To address these challenges, emerging regulatory frameworks must move beyond static compliance checks toward dynamic governance mechanisms that enforce safety constraints throughout the agent’s lifecycle, effectively operationalizing the technical safeguards described earlier. Approaches such as “Agent Constitution” offer a promising direction by embedding immutable ethical principles into the agent’s architecture, enabling pre-planning, in-planning, and post-planning safety inspections that align agent behavior with societal values [88]. Additionally, formal verification techniques adapted for non-deterministic workflows are essential to mathematically bound the risk of unsafe actions, ensuring that agents operating in critical domains like healthcare or finance adhere to strict safety specifications despite probabilistic reasoning [17]. However, a tension remains between autonomy and safeguarding; overly restrictive regulations may stifle innovation, while lax policies expose society to severe risks from deceptive or manipulated agents [152].

Future research must therefore focus on developing adaptive regulatory sandboxes that allow for the safe testing of autonomous agents under realistic adversarial conditions, alongside the creation of standardized metrics for evaluating agent trustworthiness across dimensions of safety, fairness, and robustness [153]. Ultimately, securing autonomous agency requires a holistic ecosystem where the formal methods and runtime enforcement strategies detailed previously converge with legal accountability and ethical guidelines to ensure that the deployment of LLM agents enhances rather than jeopardizes public welfare.

8 Conclusion

This subsection synthesizes the critical findings regarding prompt injection and tool-use vulnerabilities, affirming that the transition from passive language models to autonomous agents fundamentally reshapes the cybersecurity landscape. Our analysis reveals that the core vulnerability of LLM agents lies not merely in semantic misalignment but in the architectural erosion of trust boundaries between instruction and data. As demonstrated by [154], the conflation of natural language inputs with executable commands allows adversaries to bypass traditional perimeter defenses, leading to severe outcomes such as remote code execution and unauthorized data exfiltration. The survey establishes that direct prompt hijacking is increasingly complemented by sophisticated indirect injection vectors, where malicious payloads embedded in retrieved contexts—such as those exploited in [40]—manipulate agent behavior without direct user interaction, thereby expanding the attack surface to include the entire information supply chain.

A comparative evaluation of defensive architectures indicates that isolated mitigation strategies are insufficient against the compositional nature of agentic threats. While input sanitization and semantic filtering offer immediate relief, they often fail against adaptive attacks utilizing cryptographic obfuscation or multimodal triggers, as noted in [155]. Consequently, the most promising

trajectory involves holistic security frameworks that integrate runtime monitoring with formal verification. Approaches like [88] exemplify this shift by embedding immutable safety constitutions across planning and execution phases, ensuring that agents adhere to ethical constraints even under adversarial pressure. Furthermore, the deployment of multi-agent verification systems, such as [94], suggests that collaborative scrutiny among specialized agents can significantly enhance robustness against jailbreak attempts that single-model defenses miss.

However, significant challenges persist regarding the scalability and adaptability of these defenses in dynamic environments. The emergence of “infectious jailbreaks” in multi-agent societies, where a single compromised node can exponentially propagate malicious behavior across a network as shown in [13], underscores the urgent need for containment protocols that address systemic rather than individual vulnerabilities. Additionally, the potential for LLM agents to autonomously exploit zero-day vulnerabilities, evidenced by [26], highlights a precarious asymmetry where offensive capabilities may outpace defensive maturation. This reality necessitates a paradigm shift from static security postures to continuous, adversarial reinforcement learning environments.

Future research must therefore prioritize the development of probabilistic safety guarantees capable of bounding risk in non-deterministic workflows, moving beyond binary success metrics to quantify the resilience of agent trajectories under perturbation. The integration of formal methods, such as those proposed in [70], offers a rigorous pathway to verify complex tool-chaining logic before deployment. Ultimately, securing autonomous agency requires a unified ecosystem approach that harmonizes architectural isolation, advanced red-teaming frameworks like [23], and regulatory governance. Only through such interdisciplinary convergence can the community ensure that the transformative potential of LLM agents is realized without compromising the integrity of the digital and physical systems they increasingly govern.

References

- [1] Personal LLM Agents Insights and Survey about the Capability, Efficiency and Security
- [2] Can LLMs Separate Instructions From Data And What Do We Even Mean By That
- [3] Not what you’ve signed up for Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection
- [4] InjecAgent Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents
- [5] A New Era in LLM Security Exploring Security Concerns in Real-World LLM-based Systems
- [6] LLM Agents can Autonomously Hack Websites
- [7] The Instruction Hierarchy Training LLMs to Prioritize Privileged Instructions
- [8] StruQ Defending Against Prompt Injection with Structured Queries
- [9] Coercing LLMs to do and reveal (almost) anything
- [10] The Rise and Potential of Large Language Model Based Agents A Survey
- [11] Understanding the planning of LLM agents A survey
- [12] Self-Organized Agents A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization

- [13] Agent Smith A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast
- [14] AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases
- [15] Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents
- [16] LLM Platform Security Applying a Systematic Evaluation Framework to OpenAI’s ChatGPT Plugins
- [17] Formal-LLM Integrating Formal Language and Natural Language for Controllable LLM-based Agents
- [18] ToolSword Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages
- [19] Defending Against Indirect Prompt Injection Attacks With Spotlighting
- [20] MAP-REDUCE Runtime Enforcement of Information Flow Policies
- [21] Do Anything Now Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models
- [22] GPTFUZZER Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts
- [23] FuzzLLM A Novel and Universal Fuzzing Framework for Proactively Discovering Jailbreak Vulnerabilities in Large Language Models
- [24] WIPI A New Web Threat for LLM-Driven Web Agents
- [25] BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents
- [26] Teams of LLM Agents can Exploit Zero-Day Vulnerabilities
- [27] LLM Agents can Autonomously Exploit One-day Vulnerabilities
- [28] AutoAttacker A Large Language Model Guided System to Implement Automatic Cyber-attacks
- [29] The Wolf Within Covert Injection of Malice into MLLM Societies via an MLLM Operative
- [30] Flooding Spread of Manipulated Knowledge in LLM-Based Multi-Agent Communities
- [31] Mapping LLM Security Landscapes A Comprehensive Stakeholder Risk Assessment Proposal
- [32] Ignore Previous Prompt Attack Techniques For Language Models
- [33] AutoDAN Generating Stealthy Jailbreak Prompts on Aligned Large Language Models
- [34] DrAttack Prompt Decomposition and Reconstruction Makes Powerful LLM Jailbreakers
- [35] Semantic Mirror Jailbreak Genetic Algorithm Based Jailbreak Prompts Against Open-source LLMs
- [36] Neural Exec Learning (and Learning from) Execution Triggers for Prompt Injection Attacks
- [37] Chain of Attack: a Semantic-Driven Contextual Multi-Turn attacker for LLM
- [38] Foot In The Door Understanding Large Language Model Jailbreaking via Cognitive Psychology
- [39] Leveraging the Context through Multi-Round Interactions for Jailbreaking Attacks

- [40] Pandora Jailbreak GPTs by Retrieval Augmented Generation Poisoning
- [41] Phantom: General Trigger Attacks on Retrieval Augmented Language Generation
- [42] Sleeper Agents Training Deceptive LLMs that Persist Through Safety Training
- [43] Scaling Laws for Data Poisoning in LLMs
- [44] MLLM-Protector Ensuring MLLM’s Safety without Hurting Performance
- [45] Jailbreak in pieces Compositional Adversarial Attacks on Multi-Modal Language Models
- [46] Images are Achilles’ Heel of Alignment Exploiting Visual Vulnerabilities for Jailbreaking Multimodal Large Language Models
- [47] ArtPrompt ASCII Art-based Jailbreak Attacks against Aligned LLMs
- [48] AdaShield Safeguarding Multimodal Large Language Models from Structure-based Attack via Adaptive Shield Prompting
- [49] Jailbreak Vision Language Models via Bi-Modal Adversarial Prompt
- [50] CodeChameleon Personalized Encryption Framework for Jailbreaking Large Language Models
- [51] Jailbreaking Large Language Models Against Moderation Guardrails via Cipher Characters
- [52] Low-Resource Languages Jailbreak GPT-4
- [53] Eyes Closed, Safety On Protecting Multimodal LLMs via Image-to-Text Transformation
- [54] BadChain Backdoor Chain-of-Thought Prompting for Large Language Models
- [55] Play Guessing Game with LLM Indirect Jailbreak Attack with Implicit Clues
- [56] Defending Large Language Models against Jailbreak Attacks via Semantic Smoothing
- [57] Defending LLMs against Jailbreaking Attacks via Backtranslation
- [58] Universal and Transferable Adversarial Attacks on Aligned Language Models
- [59] Tree of Attacks Jailbreaking Black-Box LLMs Automatically
- [60] Jailbreaking Black Box Large Language Models in Twenty Queries
- [61] Weak-to-Strong Jailbreaking on Large Language Models
- [62] Robust Prompt Optimization for Defending Language Models Against Jailbreaking Attacks
- [63] Simulating SQL Injection Vulnerability Exploitation Using Q-Learning Reinforcement Learning Agents
- [64] Exploiting Programmatic Behavior of LLMs Dual-Use Through Standard Security Attacks
- [65] Evaluating the Exploitability of Implicit Interactions in Distributed Systems
- [66] The Meaning of Attack-Resistant Systems
- [67] Reasoning about Probabilistic Defense Mechanisms against Remote Attacks
- [68] Cognitive Overload Jailbreaking Large Language Models with Overloaded Logical Thinking
- [69] IFC Inside Retrofitting Languages with Dynamic Information Flow Control (Extended Version)

- [70] SAFECHAIN Securing Trigger-Action Programming from Attack Chains (Extended Technical Report)
- [71] LLMSecEval A Dataset of Natural Language Prompts for Security Evaluations
- [72] A Hazard Analysis Framework for Code Synthesis Large Language Models
- [73] From Text to MITRE Techniques Exploring the Malicious Use of Large Language Models for Generating Cyber Attack Payloads
- [74] You Autocomplete Me Poisoning Vulnerabilities in Neural Code Completion
- [75] An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection
- [76] SafeDecoding Defending against Jailbreak Attacks via Safety-Aware Decoding
- [77] Identifying the Risks of LM Agents with an LM-Emulated Sandbox
- [78] AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents
- [79] A New Era in Software Security Towards Self-Healing Software via Large Language Models and Formal Verification
- [80] Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risk of Language Models
- [81] SecGPT An Execution Isolation Architecture for LLM-Based Systems
- [82] When Good Components Go Bad Formally Secure Compilation Despite Dynamic Compromise
- [83] Malla Demystifying Real-world Large Language Model Integrated Malicious Services
- [84] SORRY-Bench: Systematically Evaluating Large Language Model Safety Refusal Behaviors
- [85] On Ladder Logic Bombs in Industrial Control Systems
- [86] Keeping LLMs Aligned After Fine-tuning The Crucial Role of Prompt Templates
- [87] A stochastically verifiable autonomous control architecture with reasoning
- [88] TrustAgent Towards Safe and Trustworthy LLM-based Agents through Agent Constitution
- [89] Llama Guard LLM-based Input-Output Safeguard for Human-AI Conversations
- [90] WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs
- [91] Safety Alignment Should Be Made More Than Just a Few Tokens Deep
- [92] Baseline Defenses for Adversarial Attacks Against Aligned Language Models
- [93] Representation noising effectively prevents harmful fine-tuning on LLMs
- [94] AutoDefense Multi-Agent LLM Defense against Jailbreak Attacks
- [95] LLM Self Defense By Self Examination, LLMs Know They Are Being Tricked
- [96] LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks

- [97] Beyond Good and Evil Formalizing the Security Guarantees of Compartmentalizing Compilation
- [98] Too Afraid to Drive Systematic Discovery of Semantic DoS Vulnerability in Autonomous Driving Planning under Physical-World Attacks
- [99] A Verified Information-Flow Architecture
- [100] Uninitialized Capabilities
- [101] Plug in the Safety Chip Enforcing Constraints for LLM-driven Robot Agents
- [102] Efficient Runtime Monitoring with Metric Temporal Logic A Case Study in the Android Operating System
- [103] Verifying that a compiler preserves concurrent value-dependent information-flow security
- [104] Verifying Security Policies in Multi-agent Workflows with Loops
- [105] Liquid Information Flow Control
- [106] Defending Large Language Models Against Jailbreak Attacks via Layer-specific Editing
- [107] WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models
- [108] Latent Adversarial Training Improves Robustness to Persistent Harmful Behaviors in LLMs
- [109] Mitigating Fine-tuning Jailbreak Attack with Backdoor Enhanced Alignment
- [110] Immunization against harmful fine-tuning attacks
- [111] Safety-Tuned LLaMAs Lessons From Improving the Safety of Large Language Models that Follow Instructions
- [112] LLM Defenses Are Not Robust to Multi-Turn Human Jailbreaks Yet
- [113] InferAligner Inference-Time Alignment for Harmlessness through Cross-Model Guidance
- [114] ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities
- [115] AgentBench Evaluating LLMs as Agents
- [116] τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains
- [117] Improved Techniques for Optimization-Based Jailbreaking on Large Language Models
- [118] Optimization-based Prompt Injection Attack to LLM-as-a-Judge
- [119] Red Teaming GPT-4V Are GPT-4V Safe Against Uni Multi-Modal Jailbreak Attacks
- [120] Open Sesame! Universal Black Box Jailbreaking of Large Language Models
- [121] JailbreakBench An Open Robustness Benchmark for Jailbreaking Large Language Models
- [122] HarmBench A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal
- [123] SALAD-Bench A Hierarchical and Comprehensive Safety Benchmark for Large Language Models

- [124] CyberSecEval 2 A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models
- [125] The Butterfly Effect of Altering Prompts How Small Changes and Jailbreaks Affect Large Language Model Performance
- [126] R-Judge Benchmarking Safety Risk Awareness for LLM Agents
- [127] How Johnny Can Persuade LLMs to Jailbreak Them Rethinking Persuasion to Challenge AI Safety by Humanizing LLMs
- [128] GUARD Role-playing to Generate Natural-language Jailbreakings to Test Guideline Adherence of Large Language Models
- [129] Who Validates the Validators Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences
- [130] Red Teaming Language Models with Language Models
- [131] RedAgent: Red Teaming Large Language Models with Context-aware Autonomous Language Agent
- [132] Is LLM-as-a-Judge Robust Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment
- [133] AttackEval How to Evaluate the Effectiveness of Jailbreak Attacking on Large Language Models
- [134] XSTest A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models
- [135] LLM is Like a Box of Chocolates the Non-determinism of ChatGPT in Code Generation
- [136] Multi-Agent Collaboration Harnessing the Power of Intelligent LLM Agents
- [137] Can Large Language Model Agents Simulate Human Trust Behaviors
- [138] Modularity and Openness in Modeling Multi-Agent Systems
- [139] LLM as a Mastermind A Survey of Strategic Reasoning with Large Language Models
- [140] Shadow Alignment The Ease of Subverting Safely-Aligned Language Models
- [141] Catastrophic Jailbreak of Open-source LLMs via Exploiting Generation
- [142] Fundamental Limitations of Alignment in Large Language Models
- [143] Secure Planning Against Stealthy Attacks via Model-Free Reinforcement Learning
- [144] LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks
- [145] LLM4PLC Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems
- [146] A Language for Probabilistically Oblivious Computation
- [147] Certifying LLM Safety against Adversarial Prompting
- [148] Program Actions as Actual Causes A Building Block for Accountability
- [149] A Survey of Safety and Trustworthiness of Large Language Models through the Lens of Verification and Validation

- [150] Can LLMs Keep a Secret Testing Privacy Implications of Language Models via Contextual Integrity Theory
- [151] Use Privacy in Data-Driven Systems Theory and Experiments with Machine Learnt Programs
- [152] Prioritizing Safeguarding Over Autonomy Risks of LLM Agents for Science
- [153] TrustLLM Trustworthiness in Large Language Models
- [154] Demystifying RCE Vulnerabilities in LLM-Integrated Apps
- [155] LLM Jailbreak Attack versus Defense Techniques – A Comprehensive Study